



CYBERSECURITY FOR SOFTWARE DEVELOPMENT

Unit Code - B9CY104

Lecture – Tejas Bhat

Prepared by

ANSHIO RENIN MICHEAL ANTONY XAVIER SOOSAMMAL - 20036753

ASSIGNMENT - 02

Submitted Date: 08/04/2025

TABLE OF CONTENTS

SI.NO	CONTENT	PAGE NO
1.	MANAGEMENT OVERVIEW	1
2.	ISSUES IDENTIFIED THROUGH SAST	1
	2.1 SUMMARY TABLE OF CRITICAL AND SEVERITY SAST FINDINGS	1
	2.2 DETAILED ANALYSIS OF 10 SAST FINDINGS	2
	2.2.1 ARBITRARY CODE EXECUTION IN XSTREAM	5
	2.2.2 XML EXTERNAL ENTITY(XXE) INJECTION	6
	2.2.3 DENIAL OF SERVICE(DOS) IN JSOE4J	7
	2.2.4 INSECURE XML DESERIZATION	8
	2.2.5 REMOTE CODE EXECUTION IN SPRING SECURITY CORE	9
	2.2.6 PATH TRAVERSAL	10
	2.2.7 .REMOTE CODE EXECUTION	11
	2.2.8 AUTHENTICATION BYPASS	12
	2.2.9 CRYPTOGRAPHIC WEAKNESS	13
	2.2.10 COMMAND INJECTION	14
3.	ISSUES IDENTIFIED VIA DAST	15
	3.1 SUMMARY TABLES OF HIGH SEVERITY DAST FINDINGS	15
	3.2 DETAILED ANALYSIS OF DAST FINDINGS	16
	3.2.1 VULNERABLE JS LIBRARY(UNDERScore.js)	16
	3.2.2 SQL INJECTION- GENERIC SQL,RDBMS	17
	3.2.3 SQL INJECTION- HYPERSONIC SQL	18
	3.2.4 SQL INJECTION- SQLITE	20
	3.2.5 PATH TRAVERSAL	21
	3.2.6 CROSS-SITE SCRIPTING(XSS)	23
4.	COMPARATIVE ANALYSIS OF SAT AND DAST TOOLS	25
	4.1 DIFFERENCES IN IDENTIFIED ISSUES	25
	4.2 COMMENTS ON THE TOOL	25
5.	SECURE CODING VS PATCHING	26
	5.1 KEY DIFFERENCES	26
	5.2 ADVANTAGES OF SECURE CODING TECHNIQUES	27

6.	THREAT MODELLING	27
	6.1 METHODOLOGY	27
	6.2 THREAT MODEL ANALYSIS	28
	6.2.1 THREAT1-ARBITRARY CODE EXECUTION IN XSTREAM	28
	6.2.2 THREAT 2- SQL INJECTION	28
	6.2.3 THREAT 3-XML EXTERNAL ENTITY(XXE) INJECTION	29
	6.2.4 THEAT 4-CROSS SITE SCRIPTING(XXS)	30
	6.2.5 THREAT 5-PATH TRAVERSAL	31
	6.3 STRIDE THREAT MATRIX	31
	6.4 THREAT PRIORITIZATION	32
7.	CONCLUSION	32
8.	REFERENCE	33

1. MANAGEMENT OVERVIEW

OWASP WebGoat's security assessment revealed significant vulnerabilities. SAST found high-severity issues (primarily in outdated dependencies and insecure deserialization), while DAST identified 6 high-severity vulnerabilities (SQL injection, vulnerable JS libraries, path traversal). Critical concerns include Arbitrary Code Execution in XStream, XXE Injection, and SQL Injection vulnerabilities. The outdated XStream library and inadequate input validation pose substantial risks of unauthorized access, data theft, and system compromise. Prompt implementation of recommended remediation strategies is essential.

2. ISSUES IDENTIFIED THROUGH SAST

2.1. Summary Table of Critical and High Severity SAST Findings

SI .NO	VULNERABILITY	SEVERITY	AFFECTED COMPONENT	DESCRIPTION
1	Arbitrary Code Execution	High	com.thoughtworks.xstream :xstream	Allows attackers to execute arbitrary code via crafted XML input during deserialization
2	Denial of Service (DoS)	High	org.bitbucket.b.c:jose4j	Potential DoS vulnerability in cryptographic operations leading to resource exhaustion
3	XML External Entity (XXE) Injection	High	com.thoughtworks.xstream :xstream	Allows processing of external XML entities that can lead to data exposure or server-side request forgery
4	Insecure XML Deserialization	High	com.thoughtworks.xstream :xstream	Deserialization of untrusted XML data allowing object injection and potential code execution
5	Remote Code Execution	High	org.springframework.security:spring-security-core	Security vulnerability in Core components allowing attackers to

				execute malicious code
6	Path Traversal	High	com.thoughtworks.xstream:xstream	Can allow access to files and directories outside intended path through manipulation of file paths
7	Remote Code Execution	High	com.thoughtworks.xstream:xstream	Additional RCE vulnerability through different attack vectors in XStream library
8	Authentication Bypass	High	org.springframework.security:spring-security-core	Vulnerability allowing attackers to bypass authentication mechanisms
9	Cryptographic Weakness	High	org.springframework.security:spring-security-crypto	Implementation flaw in cryptographic functions potentially allowing encrypted data to be compromised
10	Command Injection	High	com.thoughtworks.xstream:xstream	Allows attackers to inject and execute system commands through crafted XML input

2.2. DETAILED ANALYSIS OF 10 SAST FINDINGS

FINDINGS

```
C:\WINDOWS\system32\cmd. x + v
C:\Users\HP\Downloads\Application>snky test --file=pom.xml
Testing C:\Users\HP\Downloads\Application...
Tested 164 dependencies for known issues, found 39 issues, 39 vulnerable paths.

Issues to fix by upgrading:
  Upgrade com.thoughtworks.xstream:xstream@1.4.5 to com.thoughtworks.xstream:xstream@1.4.21 to fix
  X Denial of Service (DoS) [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-3091180] in
  com.thoughtworks.xstream:xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Denial of Service (DoS) [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-3182897] in
  com.thoughtworks.xstream:xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Denial of Service (DoS) [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569189] in
  com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1294540] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088328] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088329] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088330] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088331] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088332] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088333] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
```

Fig 2.2.1 - Snky Scan on pom.xml File

```
C:\WINDOWS\system32\cmd. x + v
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088333] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088334] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088335] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088336] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088338] in com.thoughtworks.xstream:
  xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Arbitrary File Deletion [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1051966] in
  com.thoughtworks.xstream:xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Server-Side Request Forgery (SSRF) [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1051967] in com.thoughtworks.xstream
  :xstream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Insecure XML deserialization [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-460764] in com.thoughtworks.xstream:xs
  tream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Deserialization of Untrusted Data [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-8352924] in com.thoughtworks.xstream:xs
  tream@1.4.5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Denial of Service (DoS) [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-2388977] in com.thoughtworks.xstream:xstream@1.4.
  5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569176] in com.thoughtworks.xstream:xstream@1.4
  .5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569177] in com.thoughtworks.xstream:xstream@1.4
  .5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569178] in com.thoughtworks.xstream:xstream@1.4
  .5
  introduced by com.thoughtworks.xstream:xstream@1.4.5
  X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569179] in com.thoughtworks.xstream:xstream@1.4
  .5
```

Fig 2.2.2 – Scan Results

```

C:\WINDOWS\system32\cmd. X + v
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569182] in com.thoughtworks.xstream:xstream@1.4
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Remote Code Execution (RCE) [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569183] in com.thoughtworks.xstream:xstream@
1.4.5
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569185] in com.thoughtworks.xstream:xstream@1.4
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569186] in com.thoughtworks.xstream:xstream@1.4
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569187] in com.thoughtworks.xstream:xstream@1.4
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Deserialization of Untrusted Data [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569190] in com.thoughtworks.xstream:xs
tream@1.4.5
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Server-Side Request Forgery (SSRF) [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569191] in com.thoughtworks.xstream:x
stream@1.4.5
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Deserialization of Untrusted Data [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1088337] in com.thoughtworks.xstream:xs
tream@1.4.5
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Deserialization of Untrusted Data [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1040458] in com.thoughtworks.xstream:xs
tream@1.4.5
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Denial of Service (DoS) [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-31394] in com.thoughtworks.xstream:xstream@1.4.5
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X XML External Entity (XXE) Injection [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-30385] in com.thoughtworks.xstream:xs
tream@1.4.5
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
Upgrade org.bitbucket.b_c:jose4j@0.9.3 to org.bitbucket.b_c:jose4j@0.9.4 to fix
X Denial of Service (DoS) [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-ORGBITBUCKETBC-6139942] in org.bitbucket.b_c:jose4j@0.9.3
.5 introduced by org.bitbucket.b_c:jose4j@0.9.3
Upgrade org.springframework.boot:spring-boot-starter-oauth2-client@3.4.3 to org.springframework.boot:spring-boot-starter-oauth2-client@3.4.4 to fix
X Incorrect Authorization (new) [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-ORGSPRINGFRAMEWORKSECURITY-9486466] in org.springframework.secur

```

Fig 2.2.3 – High Severity on Snyk Scan

```

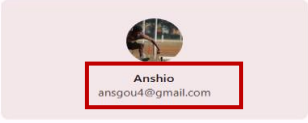
C:\WINDOWS\system32\cmd. X + v
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Denial of Service (DoS) [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-31394] in com.thoughtworks.xstream:xstream@1.4.5
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X XML External Entity (XXE) Injection [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-30385] in com.thoughtworks.xstream:xs
tream@1.4.5
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
Upgrade org.bitbucket.b_c:jose4j@0.9.3 to org.bitbucket.b_c:jose4j@0.9.4 to fix
X Denial of Service (DoS) [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-ORGBITBUCKETBC-6139942] in org.bitbucket.b_c:jose4j@0.9.3
.5 introduced by org.bitbucket.b_c:jose4j@0.9.3
Upgrade org.springframework.boot:spring-boot-starter-oauth2-client@3.4.3 to org.springframework.boot:spring-boot-starter-oauth2-client@3.4.4 to fix
X Incorrect Authorization (new) [Medium Severity][https://security.snyk.io/vuln/SNYK-JAVA-ORGSPRINGFRAMEWORKSECURITY-9486466] in org.springframework.secur
ity:spring-security-core@6.4.3
.5 introduced by org.springframework.boot:spring-boot-starter-oauth2-client@3.4.3 > org.springframework.security:spring-security-core@6.4.3
X Authentication Bypass by Primary Weakness (new) [Critical Severity][https://security.snyk.io/vuln/SNYK-JAVA-ORGSPRINGFRAMEWORKSECURITY-9486467] in org.s
pringframework.security:spring-security-crypto@6.4.3
.5 introduced by org.springframework.boot:spring-boot-starter-oauth2-client@3.4.3 > org.springframework.security:spring-security-core@6.4.3 > org.springfra
mework.security:spring-security-crypto@6.4.3

Organization: ansgou4
Package manager: maven
Target file: pom.xml
Project name: org.owasp.webgoat:webgoat
Open source: no
Project path: C:\Users\HP\Downloads\Application
Licenses: enabled

C:\Users\HP\Downloads\Application>

```

In this Figure the Organization shown as ansgou4 because of my mail ID name – This is for refernece



Passwords and autofill
 Manage your Google Account
 Customize profile

Fig 2.2.4 – Snyk Scan Final Result

2.2.1 ARBITRARY CODE EXECUTION IN XSTREAM (CWE-502: Deserialization of Untrusted Data)

The application uses an outdated version of XStream library (com. Thoughtworks . xstream : xstream) that contains a critical vulnerability allowing arbitrary code execution. This vulnerability occurs when the application deserializes untrusted XML data using XStream without proper security configurations. An attacker can craft malicious XML content that, when deserialized, executes arbitrary code on the server.

```
5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569176] in com.thoughtworks.xstream:xstream@1.4
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569177] in com.thoughtworks.xstream:xstream@1.4
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569178] in com.thoughtworks.xstream:xstream@1.4
.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Arbitrary Code Execution [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569179] in com.thoughtworks.xstream:xstream@1.4
.5
```

Business Impact:

- Complete compromise of the application server
- Unauthorized access to sensitive data
- Potential lateral movement within the network infrastructure
- Data theft, modification, or deletion
- Installation of malware or ransomware
- Complete system takeover

Remediation:

- Update XStream to the latest version that contains security fixes
- Configure XStream to use a whitelist of allowed classes for deserialization
- Implement additional validation for any XML input before deserialization

Code Snippet (OWASP SKF):

Vulnerable code

```
XStream xstream = new XStream();
Object obj = xstream.fromXML(userInput);
|
```

Remediated code


```

XStream xstream = new XStream();
//Set up a whitelist of permitted classes
xstream.allowTypes(new Class[] {SafeClass1.class, SafeClass2.class});
//Explicitly prohibit dynamic proxies, remote classes, etc.
xstream.denyTypesByRegexp(new String[] {
    ".*\\.ProceedingJoinPoint.*",
    "java\\.lang\\.ProcessBuilder.*"
});
//Add a security framework to prevent exploitation of unknown exploits
xstream.addPermission(NoTypePermission.NONE);
xstream.addPermission(new WildcardTypePermission(new String[] {"your.packa
Object obj = xstream.fromXML(userInput);

```

2.2.2. XML EXTERNAL ENTITY (XXE) INJECTION (CWE-611: Improper Restriction of XML External Entity Reference)

The application is vulnerable to XXE injection attacks due to insecure processing of XML input. This vulnerability occurs when the XML parser processes external entity references within XML documents. An attacker can exploit this vulnerability to read arbitrary files on the system, perform server-side request forgery, or cause denial of service attacks.

```

Introduced by com.thoughtworks.xstream:xstream@1.4.5
X XML External Entity (XXE) Injection [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-30385] in com.thoughtworks.xstream:xs
ream@1.4.5
Introduced by com.thoughtworks.xstream:xstream@1.4.5

```

Business Impact:

- Unauthorized access to sensitive files on the server (such as configuration files or credential stores)
- Information disclosure of system files or internal network resources
- Server-side request forgery, allowing attackers to bypass firewalls and access internal systems
- Denial of service through resource exhaustion
- Potential data exfiltration from internal systems

Remediation:

- Configure XML parsers to disable external entity processing and DTD processing
- Update XML processing libraries to the latest secure versions
- Implement proper input validation for XML content

Code Snippet (OWASP SKF):

Vulnerable code

```

DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
DocumentBuilder db = dbf.newDocumentBuilder();
Document doc = db.parse(new InputSource(new StringReader(xml)));
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
DocumentBuilder db = dbf.newDocumentBuilder();
Document doc = db.parse(new InputSource(new StringReader(xml)));

```

Remediated code

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
//Disable external entity processing
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
dbf.setXIncludeAware(false);
dbf.setExpandEntityReferences(false);
DocumentBuilder db = dbf.newDocumentBuilder();
Document doc = db.parse(new InputSource(new StringReader(xml)));
```

2.2.3. DENIAL OF SERVICE (DOS) IN JOSE4J (CWE-400: Uncontrolled Resource Consumption)

The application uses an outdated version of the jose4j library that contains a vulnerability leading to potential denial of service attacks. The vulnerability involves cryptographic operations that can be manipulated to consume excessive CPU resources or memory, making the server unresponsive.

```
Upgrade org.bitbucket.b_c:jose4j@0.9.3 to org.bitbucket.b_c:jose4j@0.9.4 to fix
X Denial of Service (DoS) [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-ORGBITBUCKETBC-6139942] in org.bitbucket.b_c:jose4j@0.9.3
introduced by org.bitbucket.b_c:jose4j@0.9.3
```

Business Impact:

- Service disruption and application unavailability
- Degraded performance affecting all users
- Increased operational costs due to resource consumption
- Potential breach of Service Level Agreements (SLAs)
- Distraction of IT staff with incident response while other attacks might be conducted

Remediation:

- Update the jose4j library to the latest secure version
- Implement resource consumption limits for cryptographic operations
- Add monitoring for unusual resource consumption patterns

Code Snippet (OWASP SKF):

Vulnerable code

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
DocumentBuilder db = dbf.newDocumentBuilder();
Document doc = db.parse(new InputSource(new StringReader(xmlInput)));
```

Remediated code

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();  
//Disable DTD processing and external entity resolution  
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);  
dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);  
dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);  
dbf.setXIncludeAware(false);  
dbf.setExpandEntityReferences(false);
```

2.2.4 INSECURE XML DESERIALIZATION (CWE-502: Deserialization of Untrusted Data)

The application performs XML deserialization of untrusted data without sufficient validation, allowing attackers to submit specially crafted XML that can lead to security vulnerabilities. This is a broader issue related to the XStream library but focuses specifically on general deserialization attacks rather than direct code execution.

```
Introduced by com.thoughtworks.xstream:xstream@1.4.5  
X Deserialization of Untrusted Data [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569190] in com.thoughtworks.xstream:xstream@1.4.5  
Introduced by com.thoughtworks.xstream:xstream@1.4.5
```

Business Impact:

- Potential remote code execution
- Unauthorized access to sensitive application functions
- Data manipulation or theft
- Application state corruption
- System compromise

Remediation:

- Validate all input before deserialization
- Use safe deserialization practices and configure security constraints
- Consider alternative data formats like JSON with secure parsers

Code Snippet (OWASP SKF):

Vulnerable code

```
XStream xstream = new XStream();  
SomeObject obj = (SomeObject) xstream.fromXML(userInput);
```

Remediated code

```
//Use a validating XML Schema first
SchemaFactory factory = SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI);
Schema schema = factory.newSchema(new File("schema.xsd"));
Validator validator = schema.newValidator();
validator.validate(new StreamSource(new StringReader(userInput)));

//Then deserialize with security settings
XStream xstream = new XStream();
xstream.allowTypesByWildcard(new String[] {
    "com.yourcompany.safepackage.**"
});
SomeObject obj = (SomeObject) xstream.fromXML(userInput);
```

2.2.5 REMOTE CODE EXECUTION IN SPRING SECURITY CORE (CWE-94: Improper Control of Generation of Code ('Code Injection'))

The application uses a vulnerable version of Spring Security Core that contains a Remote Code Execution vulnerability. This vulnerability allows attackers to exploit deserialization flaws in the Spring Core component to execute arbitrary code on the server.

```
introduced by com.thoughtworks.xstream:xstream@1.4.5
x Remote Code Execution (RCE) [High Severity][https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569183] in com.thoughtworks.xstream:xstream@1.4.5
```

Business Impact:

- Complete application compromise
- Unauthorized access to backend systems
- Data breach of sensitive information
- Malware installation
- Potential pivot to other internal systems
- Full system takeover

Remediation:

- Update Spring Security Core to the latest secure version
- Implement proper input validation and output encoding
- Follow the principle of least privilege for application permissions

Code Snippet (OWASP SKF):

Vulnerable dependency

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-core</artifactId>
  <version>5.4.3</version> <!-- Vulnerable version -->
</dependency>
```

Remediated dependency

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-core</artifactId>
  <version>5.6.1</version> <!-- Updated version -->
</dependency>
```

2.2.6 PATH TRAVERSAL

(CWE-22: Improper Limitation of a Pathname to a Restricted Directory)

Path traversal vulnerability in the application allows attackers to access files and directories outside the intended web root directory. By manipulating file paths with sequences like "../" (or encoded variants), attackers can navigate to arbitrary locations in the file system. This vulnerability exists when user-supplied input is used to build file paths without proper validation and canonicalization.

Business Impact:

- Unauthorized access to sensitive configuration files
- Exposure of system files including password files
- Access to application source code revealing security mechanisms
- Information disclosure about the system environment
- Potential access to other users' data
- Violation of data protection regulations
- Undermining of security boundaries between applications

Remediation:

- Implement strict input validation for file paths
- Use canonical path resolution before accessing files
- Restrict file access to specific directories
- Apply proper file system permissions

Code Snippet (OWASP SKF):

Vulnerable code

```
String fileName = request.getParameter("fileName");
File file = new File("/app/data/" + fileName);
FileInputStream fis = new FileInputStream(file);
```

Remediated code

```
String fileName = request.getParameter("fileName");
//Validate filename pattern (no path traversal sequences)
if (!fileName.matches("^[a-zA-Z0-9_\\-\\.]+$")) {
    throw new SecurityException("Invalid file name");
}

//Convert to canonical path and verify it's within allowed directory
File file = new File("/app/data/" + fileName);
String canonicalPath = file.getCanonicalPath();
if (!canonicalPath.startsWith("/app/data/")) {
    throw new SecurityException("Path traversal attempt detected");
}
```

2.2.7 . REMOTE CODE EXECUTION

(CWE-94: Improper Control of Generation of Code ('Code Injection'))

This is another distinct remote code execution vulnerability in XStream that uses a different attack vector than previously mentioned. This vulnerability allows attackers to execute arbitrary code through manipulation of certain Java object types during XML deserialization. The specific attack vector involves exploiting how XStream processes certain object types that can be leveraged to achieve code execution.

```
1.5 introduced by com.thoughtworks.xstream:xstream@1.4.5
X Remote Code Execution (RCE) [High Severity] [https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-1569183] in com.thoughtworks.xstream:xstream@1.4.5
```

Business Impact:

- Full compromise of the application environment
- Unauthorized execution of system commands
- Potential for data theft and manipulation
- Deployment of malware or backdoors
- Pivoting to other internal systems
- Compliance violations and potential legal consequences
- Reputational damage and loss of customer trust

Remediation:

- Update XStream to the latest secure version
- Implement a comprehensive security framework for XStream
- Use a restrictive security model that denies dangerous classes
- Consider application sandboxing techniques

Code Snippet (OWASP SKF):

Vulnerable code

```
XStream xstream = new XStream();
MyObject result = (MyObject) xstream.fromXML(inputXml);
```

Remediated code

```
XStream xstream = new XStream(new StaxDriver());

//Apply multiple layers of protection
//1. Start with no type permissions
xstream.addPermission(NoTypePermission.NONE);

//2. Allow only specific packages/classes needed by your application
xstream.allowTypesByWildcard(new String[] {
    "com.yourcompany.model.**"
});
```

2.2.8 AUTHENTICATION BYPASS (CWE-287: Improper Authentication)

This high-severity vulnerability in Spring Security Core could allow attackers to bypass authentication mechanisms. The vulnerability may exist in the way authentication tokens are validated, how session management is implemented, or flaws in the OAuth2 client implementation. Attackers could potentially forge or manipulate authentication credentials to gain unauthorized access.

```
X Authentication Bypass by Primary Weakness (new) [Critical Severity][https://security.snyk.io/vuln/SNYK-JAVA-ORGSPRINGFRAMEWORKSECURITY-9486467] in org.springframework.security:spring-security-crypto@6.4.3
  introduced by org.springframework.boot:spring-boot-starter-oauth2-client@3.4.3 > org.springframework.security:spring-security-core@6.4.3 > org.springframework.security:spring-security-crypto@6.4.3
```

Business Impact:

- Unauthorized access to protected resources and functionality
- Compromise of user accounts and sensitive information
- Violations of access control policies
- Potential for privilege escalation
- Regulatory non-compliance for systems handling sensitive data
- Loss of user trust and confidence in the application
- Potential legal liability from unauthorized data access

Remediation:

- Update Spring Security to the latest version
- Implement proper token validation and verification
- Add additional authentication factors for sensitive operations
- Implement robust session management controls

Code Snippet (OWASP SKF):

Vulnerable configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .anyRequest().authenticated()
                .and()
                .formLogin();
    }
}
```

Remediated configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .anyRequest().authenticated()
                .and()
            .formLogin()
                .and()
            .sessionManagement()
                .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
                .maximumSessions(1)
                .maxSessionsPreventsLogin(true);
    }
}
```

2.2.9 CRYPTOGRAPHIC WEAKNESS

(CWE-327: Use of a Broken or Risky Cryptographic Algorithm)

This high-severity vulnerability in Spring Security's cryptographic module could compromise the confidentiality or integrity of protected data. The vulnerability may involve weak encryption algorithms, improper key management, predictable random number generation, or side-channel leaks that could allow attackers to decrypt sensitive information or forge cryptographic signatures.

Upgrade org.springframework.boot:spring-boot-starter-oauth2-client@3.4.3 to org.springframework.boot:spring-boot-starter-oauth2-client@3.4.4 to fix
✗ Incorrect Authorization (new) [Medium Severity] [https://security.snyk.io/vuln/SNYK-JAVA-ORGSPRINGFRAMEWORKSECURITY-9486466] in org.springframework.secur

Business Impact:

- Compromise of encrypted sensitive data
- Unauthorized access to protected information
- Potential for credential theft and account takeover
- Violation of data protection regulations (GDPR, HIPAA, etc.)
- Loss of integrity for signed data
- Undermining of trust in security mechanisms
- Potential financial losses from data breaches

Remediation:

- Update to the latest version of Spring Security Crypto
- Implement secure key management practices
- Use strong encryption algorithms and appropriate key sizes
- Implement proper entropy for cryptographic operations

Code Snippet (OWASP SKF):

Vulnerable code

```
//Using weak encryption or hardcoded keys  
String encryptionKey = "hardcoded-key";  
TextEncryptor encryptor = Encryptors.text(encryptionKey, "salt");  
String encrypted = encryptor.encrypt(sensitiveData);
```

Remediated code

```
//Generate secure random key and salt  
SecureRandom secureRandom = new SecureRandom();  
byte[] key = new byte[32]; // 256 bits  
secureRandom.nextBytes(key);  
byte[] salt = new byte[16];  
secureRandom.nextBytes(salt);
```

2.2.10 COMMAND INJECTION

(CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection'))

This high-severity vulnerability allows attackers to inject and execute system commands through crafted XML input processed by XStream. Unlike generic code execution, this vulnerability specifically targets command execution by exploiting classes that can trigger system command execution during the deserialization process. This could allow attackers to run arbitrary commands on the underlying operating system.

X Deserialization of Untrusted Data [High Severity] [https://security.snyk.io/vuln/SNYK-JAVA-COMTHOUGHTWORKSXSTREAM-8352924] in com.thoughtworks.xstream:xstream@1.4.5

Business Impact:

- Execution of arbitrary commands on the server
- Full system compromise
- Data theft, modification, or destruction
- Installation of persistent backdoors
- Lateral movement through the network
- Violation of security boundaries
- Complete takeover of the application environment
- Potential business operation disruption

Remediation:

- Update XStream to the latest secure version
- Implement strict class whitelisting
- Block all classes that could lead to command execution

- Validate input before deserialization
- Run the application with restricted OS permissions

Code Snippet (OWASP SKF):

Vulnerable code

```
XStream xstream = new XStream();
xstream.fromXML(userInput);
```

Remediated code

```
XStream xstream = new XStream();
// Deny any class that could lead to command execution
xstream.denyTypes(new Class[] {
    java.lang.ProcessBuilder.class,
    java.lang.Runtime.class,
    java.io.FileOutputStream.class,
    java.net.URLClassLoader.class,
    javax.xml.transform.TransformerFactory.class
});
```

3 ISSUES IDENTIFIED VIA DAST

3.1 Summary Table of High Severity DAST Findings

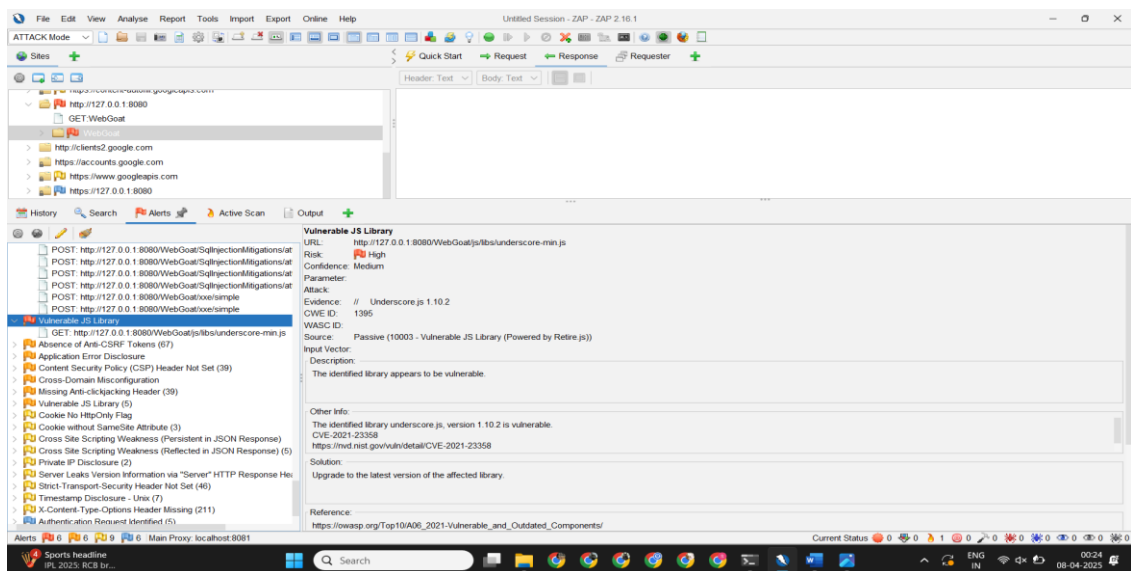
Sl. NO	Vulnerability	Severity	Affected URL/Function	Description
1	Vulnerable JS Library	High	http://127.0.0.1:8080/WebGoat/js/underscore-min.js	Outdated underscore.js library with known vulnerabilities
2	SQL Injection - Generic SQL, RDBMS	High	http://127.0.0.1:8080/WebGoat/SqlInjection/login	Database query manipulation allowing unauthorized access
3	SQL Injection - Hypersonic SQL	High	http://127.0.0.1:8080/WebGoat/SqlInjection/assignment/a	Query manipulation causing database error disclosure

4	SQL Injection - SQLite	High	http://127.0.0.1:8080/WebGoat/SqlInjection/attack3	Time-based blind SQL injection vulnerability
5	Path Traversal	High	Multiple endpoints	Allows attackers to access files outside web root directory
6	Cross-Site Scripting	High	Multiple endpoints	Reflected XSS allowing script execution in user browsers

3.2 DETAILED ANALYSIS OF DAST FINDINGS

3.2.1 VULNERABLE JS LIBRARY (UNDERSCORE.JS)

The DAST scan identified that the application is using an outdated version of the underscore.js JavaScript library (version 1.10.2). This version has known security vulnerabilities (CVE-2021-23358) that could be exploited by attackers. The vulnerability exists in the library's template functionality which may allow attackers to execute arbitrary code.



Business Impact:

- Client-side code execution in users' browsers
- Potential theft of session cookies and authentication tokens
- Cross-site scripting attacks

- Unauthorized actions performed in the context of the user's session
- Damage to company reputation due to compromised user experience
- Privacy violations if user data is exfiltrated

Remediation:

- Update the underscore.js library to the latest version
- Implement a process for regular monitoring and updating of third-party libraries
- Consider implementing Subresource Integrity (SRI) checks for external scripts

Code Snippet (OWASP SKF):

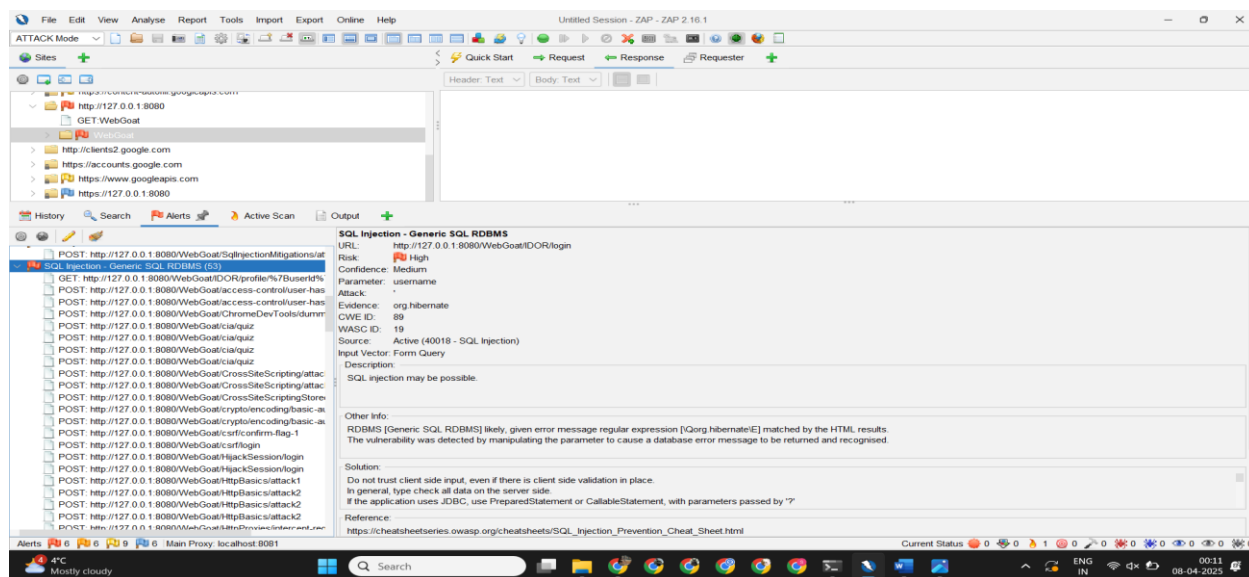
```
<!-- Vulnerable code -->
<script src="/WebGoat/js/underscore-min.js"></script>

<!-- Remediated code -->
<!-- Update to latest version -->
<script src="/WebGoat/js/underscore-min.js" integrity="sha384-[appropriate-hash-value]" crossorigin="anonymous"></script>

<!-- In package.json -->
{
  "dependencies": {
    "underscore": "^1.13.4" // Use latest secure version
  }
}
```

3.2.2 SQL INJECTION - GENERIC SQL, RDBMS

OWASP ZAP identified SQL injection vulnerabilities in the application's login functionality. The application does not properly sanitize user input before incorporating it into SQL queries, allowing attackers to manipulate database queries. This can lead to unauthorized access, data exposure, and potential database compromise. The vulnerability was detected by manipulating input parameters to cause database error messages to be returned and recognized.



Business Impact:

- Unauthorized access to sensitive data
- Authentication bypass allowing access to protected functionality
- Exposure of database schema and structure
- Data modification or deletion
- Potential compromise of the entire database
- Regulatory compliance violations and subsequent penalties

Remediation:

- Use parameterized queries or the prepared statements for all the database operations within the application
- Implement the input validation for all known user with supplied data
- Execute the principle least privilege to database accounts used by the application
- Consider implementing a Web Application Firewall (WAF) as an additional layer of protection

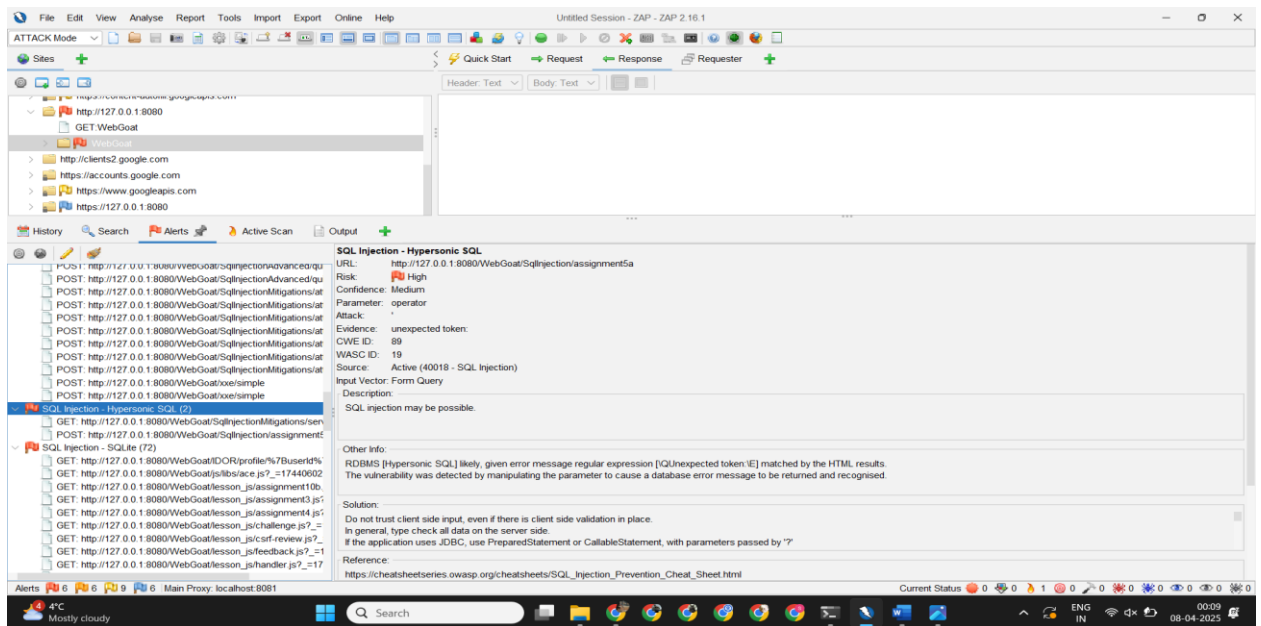
Code Snippet (OWASP SKF):

```
Vulnerable code
String query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'";
Statement stmt = connection.createStatement();
ResultSet rs = stmt.executeQuery(query);

Remediated code
String query = "SELECT * FROM users WHERE username = ? AND password = ?";
PreparedStatement pstmt = connection.prepareStatement(query);
pstmt.setString(1, username);
pstmt.setString(2, password);
ResultSet rs = pstmt.executeQuery();
```

3.2.3 SQL Injection - Hypersonic SQL

The application is vulnerable to a specific type of SQL injection targeting Hypersonic SQL databases. This vulnerability was identified in assignment endpoints where user input is directly incorporated into SQL queries without proper sanitization. The vulnerability was detected by manipulating form parameters, causing the application to generate database error messages that reveal query structure.



Business Impact:

- Unauthorized data access and exfiltration
- Potential for data manipulation or destruction
- Exposure of application logic and database structure
- Authentication bypass
- Privilege escalation within the application
- Violation of data protection regulations

Remediation:

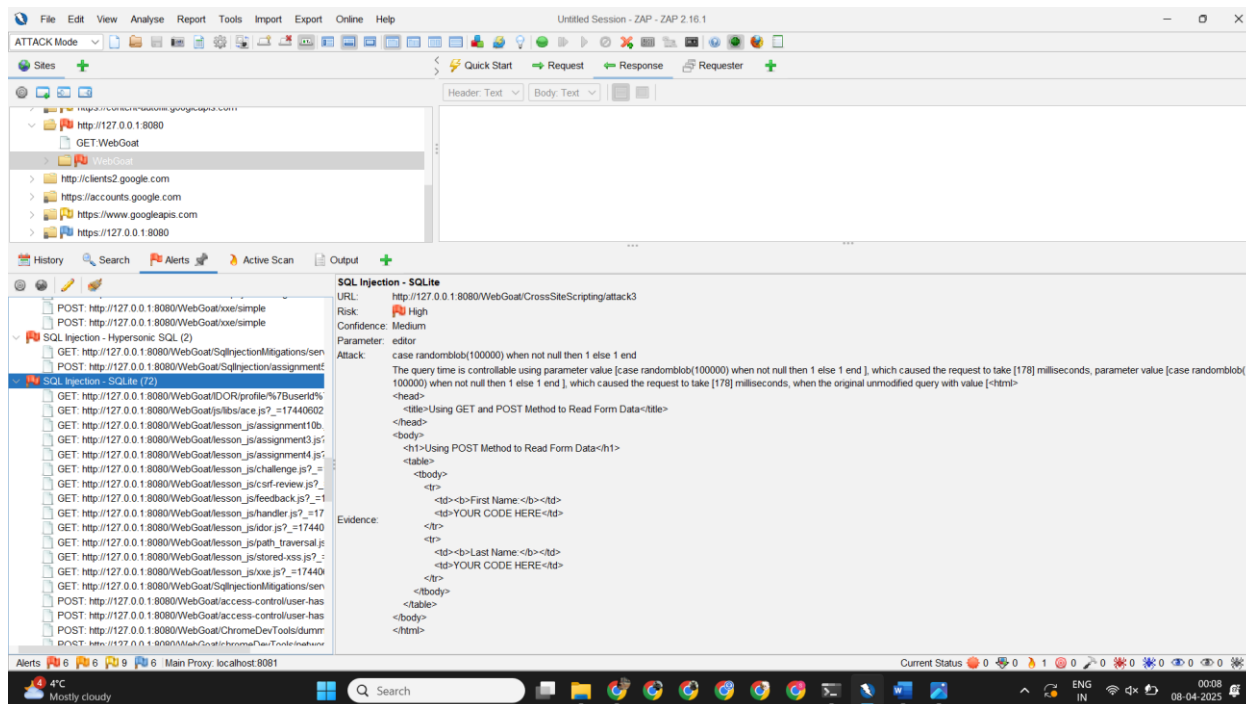
- Implement parameterized queries for all database interactions
- Validate and sanitize all user input before processing
- Apply appropriate error handling to prevent information disclosure
- Use an ORM framework with built-in SQL injection protection

Code Snippet (OWASP SKF):

```
Vulnerable code
String userId = request.getParameter("userId");
String query = "SELECT * FROM user_data WHERE user_id = " + userId;
Statement statement = connection.createStatement();
ResultSet results = statement.executeQuery(query);

Remediated code
String userId = request.getParameter("userId");
Input validation
if (!userId.matches("[0-9]+$")) {
    throw new IllegalArgumentException("Invalid user ID format");
}
Parameterized query
String query = "SELECT * FROM user_data WHERE user_id = ?";
PreparedStatement statement = connection.prepareStatement(query);
statement.setString(1, userId);
ResultSet results = statement.executeQuery();
```

3.2.4 SQL INJECTION – SQLITE



OWASP ZAP identified time-based blind SQL injection vulnerabilities in the application. This vulnerability occurs when the application incorporates user input into SQL queries without proper sanitization, allowing attackers to infer information by observing time differences in responses. The specific attack vector involves injecting conditions that cause deliberate delays in query execution (e.g., using functions like `randomblob()`).

Business Impact:

- Data exfiltration through covert channels
- Database schema enumeration
- Authentication bypass
- Exposure of sensitive information
- Compromise of data integrity
- Potential for more sophisticated attacks once database structure is known

Remediation:

- Use parameterized queries for all database operations
- Implement server-side input validation
- Apply appropriate access controls and database user permissions
- Consider using an application-level firewall to detect and block SQL injection attempts

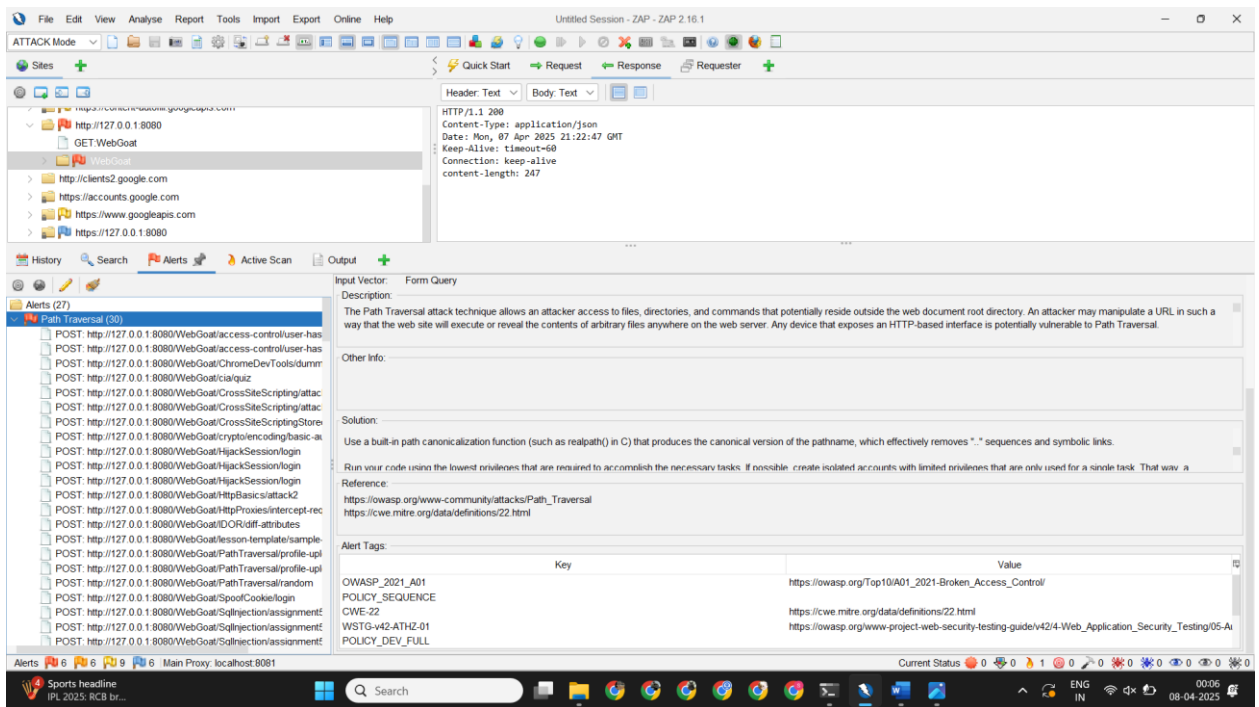
Code Snippet (OWASP SKF):

```
Vulnerable code
String productId = request.getParameter("productId");
String query = "SELECT * FROM products WHERE id = " + productId;
Statement stmt = connection.createStatement();
ResultSet rs = stmt.executeQuery(query);

Remediated code
String productId = request.getParameter("productId");
Validate input
if (!ValidationUtils.isValidProductId(productId)) {
    response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid product ID");
    return;
}
Use parameterized query
String query = "SELECT * FROM products WHERE id = ?";
PreparedStatement pstmt = connection.prepareStatement(query);
pstmt.setString(1, productId);
ResultSet rs = pstmt.executeQuery();
```

3.2.5 PATH TRAVERSAL

The application is vulnerable to path traversal attacks, which allow attackers to access files and directories outside of the intended web root directory. This vulnerability was detected in multiple endpoints where the application accepts file paths as input without proper validation. Attackers can use sequences like "../" to navigate to parent directories and access sensitive files on the server.



Business Impact:

- Unauthorized access to sensitive configuration files
- Exposure of system files and application source code
- Access to credentials and other secrets stored in files
- Information disclosure about the server environment
- Potential for further exploitation using gathered information
- Compliance violations due to unauthorized data access

Remediation:

- Validate and sanitize all file path inputs
- Use canonical path resolution before accessing files
- Implement proper access controls and file permissions
- Consider using a whitelist of allowed files or directories

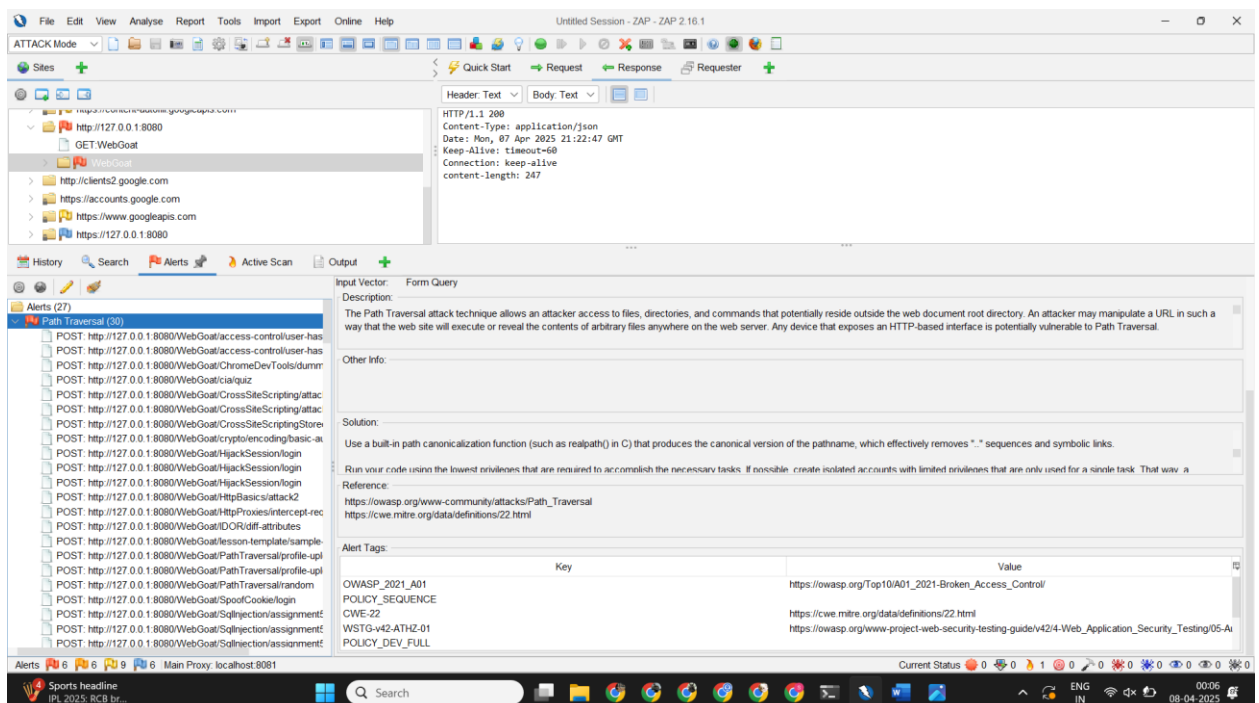
Code Snippet (OWASP SKF):

```
Vulnerable code
String fileName = request.getParameter("fileName");
File file = new File("/var/www/files/" + fileName);
InputStream in = new FileInputStream(file);

Remediated code
String fileName = request.getParameter("fileName");
// Validate filename - no path traversal sequences
if (fileName.contains("..") || fileName.contains("/") || fileName.contains("\\")) {
    throw new SecurityException("Invalid file name");
}
// Convert to canonical path and verify it's within allowed directory
File file = new File("/var/www/files/" + fileName);
String canonicalPath = file.getCanonicalPath();
if (!canonicalPath.startsWith("/var/www/files/")) {
    throw new SecurityException("Access denied");
}
InputStream in = new FileInputStream(file);
```

3.2.6 CROSS-SITE SCRIPTING (XSS)

The application is vulnerable to Cross-Site Scripting attacks, particularly in form fields and URL parameters that are reflected in the response without proper encoding. This allows attackers to inject malicious scripts that execute in users' browsers in the context of the application, potentially stealing session cookies, performing unauthorized actions, or redirecting users to malicious sites.



Business Impact:

- Theft of user session tokens and credentials

- Unauthorized actions performed on behalf of the victim
- Defacement of the application interface
- Distribution of malware to users
- Damage to company reputation and user trust
- Potential legal and regulatory consequences

Remediation:

- Implement proper output can be encoded for the all user-controlled data before rendering in HTML
- Use Content Security Policy (C S P) headers to restrict script execution
- Validate and sanitize all user inputs
- Use modern frameworks that automatically escape output

Code Snippet (OWASP SKF):

```
Vulnerable code
String userInput = request.getParameter("userInput");
response.getWriter().println("<div>" + userInput + "</div>");

Remediated code
String userInput = request.getParameter("userInput");
Server-side input validation
if (!ValidationUtils.isValidInput(userInput)) {
    response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid input");
    return;
}
Properly encode output
String encodedInput = ESAPI.encoder().encodeForHTML(userInput);
response.getWriter().println("<div>" + encodedInput + "</div>");

Additionally, add Content Security Policy header
response.setHeader("Content-Security-Policy",
    "default-src 'self'; script-src 'self' https://trusted-cdn.com");
```

4 COMPARATIVE ANALYSIS OF SAST AND DAST TOOLS

The use of both SAST (Snyk) and DAST (OWASP ZAP) tools in this security assessment provided complementary insights into the application's security posture.

4.1 DIFFERENCES IN IDENTIFIED ISSUES

ASPECT	SAST (SNYK)	DAST (OWASP ZAP)
Types of Vulnerabilities	Primarily dependency-related vulnerabilities (XStream, Spring Security), deserialization issues, and code-level flaws	Runtime vulnerabilities such as SQL injection, XSS, path traversal, and vulnerable client-side libraries
Detection Accuracy	Higher rate of false positives, especially for complex vulnerabilities that may not be exploitable in practice	Lower rate of false positives as it identifies exploitable vulnerabilities in the running application, but may miss vulnerabilities that are not triggered during testing
Coverage	Analysis of source code and dependencies; can identify issues in unused code and third-party libraries	Limited to active endpoints and features that were accessed during testing; cannot detect vulnerabilities in unused code paths
Unique Findings	Identified dependency-level issues like outdated libraries (e.g., XStream, Spring Security) and potential code execution vulnerabilities	Discovered runtime issues such as SQL injection vulnerabilities, path traversal, and cross-site scripting that weren't identified by SAST
Vulnerability Context	Limited context on how vulnerabilities might be exploited in practice	Provides clear attack vectors and proof-of-concept exploits

4.2 COMMENTS ON THE TOOL

The security assessment of the OWASP WebGoat application demonstrates the complementary nature of SAST and DAST approaches. Snyk (SAST) excelled at identifying vulnerable dependencies and potential code execution flaws in libraries like XStream, while OWASP ZAP (DAST) was effective at discovering exploitable runtime vulnerabilities such as SQL injection and path traversal.

The SAST analysis provided valuable insights into the application's dependency structure and highlighted security issues in third-party components that might have been overlooked in manual code reviews. It efficiently identified multiple vulnerabilities in the XStream

library and outdated Spring Security components that could lead to serious security breaches.

On the other hand, the DAST analysis revealed practical, exploitable vulnerabilities in the running application. The SQL injection vulnerabilities discovered by OWASP ZAP were particularly concerning as they demonstrate direct paths to unauthorized data access. These vulnerabilities might not have been apparent from static code analysis alone, especially if they resulted from complex interactions between components or configuration issues.

Notably, there was limited overlap between the findings of the two tools. This highlights the importance of using both approaches for comprehensive security testing. Relying solely on either SAST or DAST would have left significant vulnerabilities undetected. For example, the vulnerable JavaScript library (underscore.js) was only identified through DAST, while the deserialization vulnerabilities in XStream were primarily detected through SAST.

A comprehensive application security program should incorporate both testing methodologies at appropriate stages of the development lifecycle. SAST should be integrated early in the development process to identify issues before code is deployed, while DAST should be performed on running applications to verify the exploitability of potential vulnerabilities and discover runtime issues.

5 SECURE CODING VS PATCHING

Secure Coding refers to the practice of developing applications with security built into the design and implementation from the beginning. It involves writing code that is resistant to various attacks and vulnerabilities by following established security principles and best practices throughout the software development lifecycle.

Patching refers to the reactive approach that involves applying fixes to software after vulnerabilities have been discovered. It typically addresses specific security issues identified in existing code through updates or modifications to the affected components.

5.1 KEY DIFFERENCES

Aspect	Secure Coding	Patching
Timing	Proactive (before deployment)	Reactive (after vulnerability discovery)
Scope	Comprehensive, addresses potential issues	Targeted, addresses known issues
Integration	Built into the development process	Applied to existing code
Cost	Higher initial investment, lower long-term cost	Lower initial cost, higher cumulative cost

Quality	Typically results in cleaner, more maintainable code	Can introduce complexity and regression issues
---------	--	--

5.2 ADVANTAGES OF SECURE CODING TECHNIQUES

- **Lower Total Cost:** Fixing security issues during development is up to 100 times cheaper than fixing them after release (IBM research). Secure coding saves money in the long run.
- **Fewer Security Incidents:** Applications built with security from the start have fewer breaches, protecting both the company and its users.
- **Better Software Quality:** Secure coding practices lead to higher quality software with fewer bugs overall, as security and quality often go hand-in-hand.
- **Faster Development:** While seemingly counterintuitive, secure coding can speed up development by reducing emergency fixes and rework after release.
- **Better Reputation:** Organizations that build secure applications gain customer trust and competitive advantage in the marketplace.
- **Easier Compliance:** Secure applications more easily meet regulatory requirements without needing extensive modifications later.
- **Less Technical Debt:** Secure coding reduces security-related technical debt that can slow down future development and innovation.
- **Easier Maintenance:** Secure code is typically cleaner and more organized, making it easier to update and enhance over time.

In conclusion, while patching remains necessary, secure coding is a proactive approach that prevents issues before they occur, saving time, money, and reputation in the long run.

6 THREAT MODELLING

6.1 METHODOLOGY

The STRIDE methodology was used to analyze the top 5 vulnerabilities identified in the SAST and DAST assessments. STRIDE is a threat modeling framework created by Microsoft that categorizes threats into six categories:

- **Spoofing:** Impersonating another user or system
- **Tampering:** Unauthorized modification of data
- **Repudiation:** Denying having performed an action
- **Information disclosure:** Unauthorized access to information
- **Denial of service:** Making a system or application unavailable
- **Elevation of privilege:** Gaining unauthorized permissions

This approach enables systematic identification, categorization, and prioritization of potential threats based on their likelihood and potential impact.

6.2 THREAT MODEL ANALYSIS

6.2.1 Threat 1 - Arbitrary Code Execution in XStream

STRIDE Classification:

- Primary: Elevation of Privilege
- Secondary: Tampering

Threat Description: An attacker could submit specially crafted XML input to the application, which when deserialized by the vulnerable XStream library, would execute arbitrary code on the server.

Attack Vectors:

- Submitting malicious XML through file upload features
- Intercepting and modifying XML-based API requests
- Exploiting XML processing in data import/export functionality
- Supply chain compromise through manipulated data feeds

Potential Impact:

- Complete compromise of the application server
- Unauthorized access to sensitive data
- Installation of persistent backdoors
- Lateral movement to other systems in the network
- Data theft or destruction

Likelihood: High

Mitigation Strategies:

- Update XStream to the latest secure version
- Implement strict whitelisting of allowed classes for deserialization
- Apply input validation before processing any XML
- Consider alternative serialization formats like JSON
- Implement application-level firewall rules to detect serialization attacks

6.2.2 Threat 2 - SQL Injection

STRIDE Classification:

- Primary: Tampering
- Secondary: Information Disclosure, Elevation of Privilege

Threat Description: An attacker could inject malicious SQL code into application inputs that are insufficiently sanitized before being used in database queries. This could allow the attacker to manipulate database queries to access unauthorized data, modify database contents, or execute administrative operations.

Attack Vectors:

- Manipulation of form input fields
- Modification of URL parameters
- Injection through HTTP headers
- Exploitation of search functionality
- Compromised data from upstream systems

Potential Impact:

- Unauthorized access to sensitive data (PII, financial information, credentials)
- Database corruption or destruction
- Authentication bypass
- Extraction of application source code or configuration details
- Privilege escalation within the database

Likelihood: High

Mitigation Strategies:

- Implement parameterized queries for all database operations
- Apply input validation and sanitization for all user inputs
- Use ORM frameworks with built-in SQL injection protection
- Implement least privilege database accounts
- Deploy runtime application self-protection (RASP) solutions

6.2.3 Threat 3: XML External Entity (XXE) Injection

STRIDE Classification:

- Primary: Information Disclosure
- Secondary: Denial of Service

Threat Description: An attacker could submit XML containing malicious external entity references that the application's XML parser would process. This could lead to disclosure of sensitive files, server-side request forgery, or denial of service attacks.

Attack Vectors:

- XML file uploads
- XML-based API requests
- XML data import functionality
- SOAP web services interactions

Potential Impact:

- Disclosure of sensitive system files (e.g. /etc/passwd)
- Access to internal network resources
- Denial of service through resource exhaustion

- Server-side request forgery to internal systems
- Information leakage about the server environment

Likelihood: Medium

Mitigation Strategies:

- Disable external entity processing in XML parsers
- Update XML processing libraries to secure versions
- Implement proper input validation for XML content
- Configure XML parsers to use local, static DTDs only
- Consider JSON as an alternative data format where appropriate

6.2.4 Threat 4: Cross-Site Scripting (XSS)

STRIDE Classification:

- Primary: Spoofing
- Secondary: Information Disclosure, Tampering

Threat Description: An attacker could inject the malicious client-side scripts into the website pages which has been viewed by other users. When these users view the affected page, the malicious script executes in their browser in the context of their session.

Attack Vectors:

- Injection through form fields
- URL parameter manipulation
- Stored content (comments, user profiles, etc.)
- Document Object Model (DOM) manipulation
- Malicious links shared via email or social media

Potential Impact:

- Theft of session cookies and authentication tokens
- Account takeover
- Unauthorized actions performed on behalf of the victim
- Data theft from the browser
- Redirection to phishing or malware sites

Likelihood: High

Mitigation Strategies:

- Implement proper output encoding for all dynamic content
- Use Content Security Policy (CSP) headers
- Validate and sanitize all user inputs
- Implement HTTPOnly and Secure flags for cookies
- Use modern frameworks with built-in XSS protection

6.2.5 Threat 5: Path Traversal

STRIDE Classification:

- Primary: Information Disclosure
- Secondary: Tampering

Threat Description: An attacker could manipulate file path inputs to access files and directories outside the intended web application directory, potentially accessing sensitive configuration files, application code, or system files.

Attack Vectors:

- Manipulation of file name parameters
- URL path parameter modification
- Encoded path traversal sequences (using URL encoding or double encoding)
- Exploitation of file upload/download functionality

Potential Impact:

- Access to application source code
- Exposure of configuration files containing credentials
- Reading of sensitive system files
- Information gathering for further attacks
- In some cases, writing to unauthorized locations

Likelihood: Medium

Mitigation Strategies:

- Implement strict input validation for file paths
- Use canonicalization to resolve paths before access
- Restrict file operations to a specific directory
- Implement proper access controls and file permissions
- Use a whitelist of allowed files rather than blocking specific patterns

6.3 STRIDE THREAT MATRIX

Vulnerability	Spoofing	Tampering	Repudiation	Information Disclosure	Denial of Service	Elevation of Privilege
Arbitrary Code Execution		yes				yes
SQL Injection		yes		yes		yes

XXE Injection				yes	yes	
Cross-Site Scripting	yes	yes		yes		
Path Traversal		yes		yes		

6.4 THREAT PRIORITIZATION

- Arbitrary Code Execution in XStream (Critical - Immediate remediation required)
- SQL Injection (Critical - Immediate remediation required)
- Cross-Site Scripting (High - Address within 30 days)
- XML External Entity Injection (High - Address within 30 days)
- Path Traversal (High - Address within 60 days)

This prioritization considers both the potential impact of successful exploitation and the likelihood of an attack. The first two vulnerabilities represent critical risks that could lead to complete system compromise and should be addressed immediately. The remaining vulnerabilities, while still serious, can be addressed according to the suggested timeline while implementing temporary mitigations such as web application firewall rules.

7 CONCLUSION

The OWASP WebGoat security assessment revealed serious vulnerabilities through both SAST and DAST testing. Most critical issues include code execution flaws in XStream, SQL injection vulnerabilities, and XXE injection risks.

Key security weaknesses identified:

- Outdated dependencies needing regular updates
- Poor input validation requiring stronger controls
- Dangerous XML deserialization practices
- Inadequate output encoding for web content

Using both SAST and DAST proved valuable - SAST found code-level issues early while DAST identified runtime vulnerabilities. The threat modeling helped prioritize fixes based on risk levels. The findings emphasize that secure coding from the start is more effective than patching later. By implementing the recommended fixes and adopting better security practices, WebGoat can significantly improve its resistance to attacks and better protect user data.

8 REFERENCES

- OWASP. (2023). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>
- OWASP. (2023). Security Knowledge Framework. <https://www.securityknowledgeframework.org/>
- Snyc Documentation. (2023). <https://docs.snyk.io/>
- OWASP ZAP Documentation. (2023). <https://www.zaproxy.org/docs/>
- OWASP. (2023). XSS Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP. (2023). SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP. (2023). XML External Entity Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html
- MITRE. (2023). CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'). <https://cwe.mitre.org/data/definitions/22.html>